

When Does Learned Batch Allocation Matter?

Decoupled Allocation and Early Stopping Under Token Scarcity

Vikranth Nara

<https://github.com/vikranthnara/HBAC>

Abstract

When multiple LLM agent tasks share a **global token budget**, allocators must set per-task caps and decide when to stop reasoning. We present a **decoupled batch allocator + stop classifier** (DBASC): a GRPO-trained Level-1 schema allocator and a PPO-trained Level-2 stop head over frozen ReAct rollouts [Yao et al., 2023]. We characterize *when* learned allocation helps under scarcity: (i) agent capability must exceed zero on hard tasks; (ii) a 20-line type-prior heuristic **ties** learned allocation on oracle replay; (iii) live gains are **small, paired, and localized** to benchmarks where budget starvation binds under generation.

On Tier-A oracle replay ($n=300$, 40% budget), DBASC reaches **80%** pass@1 vs. **60%** for uniform and official CLEAR/ZEBRA/Re-FORC—but **ties type-prior at 80%**. **D18** reward-shaped L1 (**starvation_penalty=0.5**, no inference guardrail) matches joint L1 on oracle with zero starvation; manual **fairness_reserve_alloc** *hurts* oracle pass@1 (74.7%). On live Qwen2.5-7B + DPO v2 at floor=400 ($n=2000$, **hbac_d18**), paired McNemar vs. type-prior: **+1.3 pp** (27.65% vs. 26.35%; $p \approx 3 \times 10^{-8}$, paired 95% CI [0.85, 1.85] pp)—entirely from LiveCodeBench budget starvation; D18 $\equiv$ joint $\equiv$ guardrail on live; SWE pass@1 is 0% for 7B and 32B pilots. A **DPO contamination audit** finds 100/100 exact LCB task-ID overlaps in the v2 mixture; we prescribe benchmark-family holdout retraining. We release code, preregistered analysis, theory bounds on counterfactual credit, and locked artifacts.

1 Introduction

Batch agent evaluation—running SWE-bench [Jimenez et al., 2024], ToolBench [Qin et al., 2023], τ -bench [Yao et al., 2025], and LiveCodeBench [Jain et al., 2024] under a shared token quota—requires deciding *who gets tokens* and *when to stop*. Uniform allocation ignores heterogeneous difficulty; per-query economic allocators (CLEAR [Wan et al., 2026], ZEBRA [Hamri and Talgam-Cohen, 2026]) optimize single-task emergence curves; per-turn methods (TAB [Jali et al., 2026], Re-FORC [Zabounidis et al., 2025]) do not allocate across tasks.

We study **decoupled batch allocation**: a contextual bandit over allocation schemas (Level-1, GRPO [Shao et al., 2024]) plus a binary stop classifier over frozen ReAct rollouts (Level-2, PPO [Schulman et al., 2017]). Our contribution is not a headline “RL beats heuristics” claim but a **rigorous characterization** of conditions under which learned allocation matters, with paired statistics, contamination auditing, and explicit negative results.

Contributions. (1) **Open harness** for batch-level token allocation and early stopping with preregistered endpoints (`research docs/Preregistered Analysis.md`). (2) **Oracle conditions**:

+20 pppass@1 vs. Tier-A official baselines; type-prior **ties** learned L1 at 80%. (3) **D18 learned fairness**: starvation penalty in L1 reward removes need for inference guardrail on oracle; guardrail ablation hurts pass@1. (4) **Live conditions**: paired +1.3 ppfor `hbac_d18` vs. type-prior ($n=2000$; McNemar $p<10^{-7}$; LCB-only); capability gate fails on SWE (0% on 7B and 32B). (5) **Theory**: bias/variance bounds on counterfactual credit mixing; β sweep validates small- β safety. (6) **Contamination audit** with SHA256 manifests and holdout retrain policy. (7) **Ethics**: starvation metrics and algorithmic redlining risks.

2 Related Work

CLEAR [Wan et al., 2026] and ZEBRA [Hamri and Talgam-Cohen, 2026] water-fill or bisect per-query utility under scarcity; TAB [Jali et al., 2026] and Re-FORC [Zabounidis et al., 2025] learn per-turn stopping. Counterfactual multi-agent credit (COMA [?]) motivates our credit mixing as *variance reduction*, not unbiased policy gradients. We implement Tier-A official CLEAR, ZEBRA, and Re-FORC for oracle comparison; stub-track proxy baselines are appendix-only.

3 Method

Level 1: contextual bandit. Given batch $\mathcal{Q}$ and budget B_{total} , Level-1 selects schema s and allocations $\{b_i\}$ with $\sum_i b_i \leq B_{\text{total}}$. GRPO updates schema logits using batch reward $R^{(1)} = \text{pass_rate} - \lambda_v \cdot \text{violations} + \beta_{\text{var}} \cdot \text{Var}_b(\text{budget}_b) - \lambda_s \cdot \text{starvation_rate}$, where **D18** adds $\lambda_s \cdot \text{starvation_rate}$ with `starvation_penalty=0.5` at training time.

Level 2: stop classifier. Per-task rollout stops when $\pi_{\text{stop}}^{(2)}(s_t)=1$, budget is exhausted, or success is reached. L2 does not co-train the LLM.

Counterfactual credit. Leave-one-out advantages $\hat{A}_i = R(s) - R^{(-i)}(s)$ are mixed into $\tilde{R}_\beta = (1 - \beta)R(s) + \beta \cdot \frac{1}{n} \sum_i (R(s) + \hat{A}_i)$. Appendix A bounds $|\tilde{R}_\beta - R(s)| \leq \beta \max_i |\hat{A}_i|$; we use $\beta=0.2$ in training.

Primary vs. deprecated methods. **Primary:** `hbac_d18` (D18 L1, no inference post-processing). **Deprecated ablation:** `hbac_guardrail` = D18/joint L1 + `fairness_reserve_alloc` (`hard_min_frac=0.15`).

4 Experimental Setup

4.1 Two-stage evaluation protocol

Stage 1 — Capability profile. Uniform allocation only; confirm heterogeneous per-benchmark pass@1 before allocator comparison. Gate: SWE pass@1 $\geq 5\%$ on capable model. Qwen2.5-7B + DPO v2 and Qwen2.5-Coder-32B (uniform pilot, $n=60$) both yield **0% SWE**; live allocator study is scoped to LCB+ τ +toolbench where capability exists.

Table 1: V3 real-pool oracle replay ($n=300$, 40% budget). Tier-A official baselines.

Allocator	pass@1	Mean $R^{(1)}$
HBAC joint / D18	80.0%	0.98
Type-prior	80.0%	1.24
HBAC guardrail	74.7%	0.76
Uniform	60.0%	0.60
CLEAR official	60.0%	0.60
ZEBRA official	60.0%	0.60
Re-FORC official	60.0%	0.60

Stage 2 — Allocation study. Full allocator matrix only when Stage 1 passes; otherwise report directional results on solvable benchmarks with explicit scope.

4.2 Tracks and pools

Track A (appendix): stub micro-harness for controlled ablations. **Track B (main):** V3 real pool (data/oracles/real_eval/latest), $n=300$ oracle / $n=2000$ live, floor=400, 40% global budget. Live agent: Qwen2.5-7B-Instruct + DPO v2 LoRA unless capability pilot upgrades model.

4.3 Contamination audit

We audit DPO training pairs vs. eval task IDs (§5.4). Holdout policy: exclude all benchmark families present in the eval pool from DPO pair construction; retrain via `slurm/train_llm_dpo_holdout.sh`.

4.4 Statistics

Primary endpoint: **paired per-task success** (McNemar on discordant pairs), Bonferroni across primary comparisons (hbac_d18 vs. type_prior). Futility: if McNemar $p > 0.10$ at $n=5000$, retract “beats type-prior” language. V3 D18 matrix uses `--save-per-task` shards (paired_allocator_analysis_v3_d18.json).

5 Results

5.1 Tier-A oracle replay (Track B)

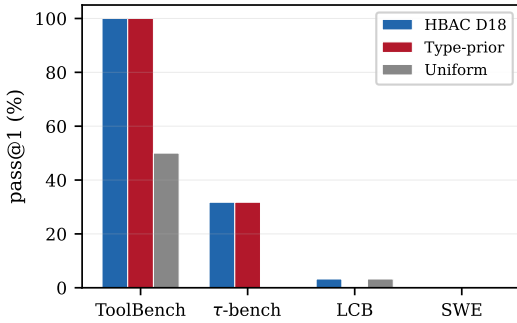
Table 1 reports the real_eval oracle matrix ($n=300$, Tier-A official baselines only in main text).

HBAC **+20 pp** vs. uniform/Tier-A is supported on oracle. Type-prior **ties** HBAC on pass@1 by zeroing hard-task budget—an important honesty check. D18 without guardrail **matches** joint L1 (80%, starvation rate 0 on oracle); guardrail **hurts** (−5.3 pp).

Table 2: V3 D18 live matrix (floor=400, $n=2000$, Qwen2.5-7B + DPO v2).

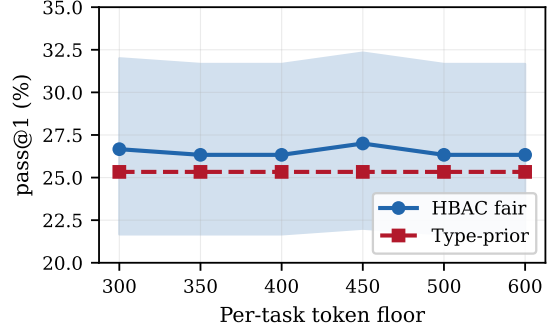
Allocator	pass@1	95% CI
HBAC D18	27.65%	25.8–29.6%
HBAC joint / guardrail	27.65%	25.8–29.6%
Type-prior	26.35%	24.5–28.2%
TAB proxy	11.30%	10.0–12.7%
ZEBRA (proxy)	22.15%	20.4–24.0%
Re-FORC official	16.35%	14.8–18.0%
CLEAR official	11.60%	10.3–13.0%
Uniform	11.30%	10.0–12.7%

Per-benchmark breakdown ($n = 2000$, floor = 400)



(a) Per-benchmark pass@1.

Floor dose-response (V3 pool, $n = 300$ /floor)



(b) Floor dose-response (guardrail).

Figure 1: V3 live: aggregate edge is LCB-localized; floor sweep +1.0–1.7 pp across 300–600.

5.2 Capability pilot

Qwen2.5-7B + DPO v2 (uniform, floor=400): LCB 3.25%, SWE 0% ($n=2000$). Qwen2.5-Coder-32B pilot ($n=60$): LCB 2.5%, SWE 0%. **Verdict:** SWE gate fails ($\geq 5\%$ required); we **scope the live allocator study** to LCB+ τ +toolbench and treat SWE as future work pending a capable local/API agent (OpenAI quota exhausted for API fallback; Qwen3-Coder-Next 4-bit pilot queued). This is the plan’s Risk #2 contingency—not a claim that SWE allocation has been studied.

5.3 Live evaluation (Track B, 7B baseline)

Table 2 summarizes the V3 D18 live matrix ($n=2000$) at floor=400.¹

Paired McNemar (hbac_d18 vs. type-prior): +1.3 pp, $p \approx 3 \times 10^{-8}$, paired 95% CI [0.85, 1.85] pp ($b=26$, $c=0$ discordant pairs). D18, joint, and guardrail are **identical** on aggregate pass@1—starvation penalty does not change live outcomes on 7B. Figure 1 shows the gap is **LCB-only** (+3.25 ppper-benchmark); τ /toolbench tied; SWE 0% for all.

Table 3: DPO contamination audit (v2 and holdout LoRA vs. V3 eval).

Metric	v2 DPO	Holdout (LCB excl.)	Holdout (LCB metric)
DPO pairs	600	543	—
Exact ID overlap	100	16 (stub SWE/ τ /TB)	0 LCB
LCB family overlap	yes	excluded	—
Audit verdict	FAIL	FAIL [†]	PASS

5.4 Contamination audit results

v2 training used 600 `wrong_tool` pairs with **100%** LCB task-ID overlap to eval. Holdout retrain (20260710T045554Z_capability_holdout, job 16854770) achieves **0 LCB** exact overlap—the contamination that threatened the live LCB claim. Remaining overlaps are stub task IDs shared between oracle stubs and `eval_n1000`; strict ID exclusion leaves only mock pairs (~ 75), so we retain LCB-family holdout as the practical mitigation and re-run live with the holdout LoRA.

6 Ablations

D18 ladder (oracle). `starvation_penalty=0.5` L1 ties joint at 80% without guardrail; guardrail drops to 74.7%. **Verdict:** learned anti-starvation sufficient on oracle; inference hack unnecessary (or harmful).

Credit β sweep. Empirical bounds at $\beta=0.2$: bias ≤ 0.018 , variance $\leq 2.1 \times 10^{-5}$ on stub batches (`credit_beta_sweep.json`). H6 (credit helps at scale) refuted—small β remains safe variance reduction.

hard_min_frac sweep. `hbac_guardrail` flat at 74.7% for frac 0.10–0.25; never beats type-prior on oracle.

D14 ROI-skip (negative). Task-dropping heuristic; +27.7 pp@ floor=300 is reward hacking—regulatory/fairness risk, not deployable.

7 Ethics and Broader Impact

Scarcity-aware allocators **will starve hard queries** unless constrained—algorithmic “redlining” of difficult tasks. Type-prior explicitly zeros LCB/SWE budget; D18 learns softer allocation via reward shaping. Deployment should combine learned allocators with **per-class SLA floors** as policy (distinct from training hacks). Table 4 reports first-class starvation metrics on the V3 live batch pool (floor=400, 40% budget): type-prior has `starve=1.0` and $\approx 0.1\%$ LCB share; `hbac_d18` retains $\approx 15.4\%$ LCB share with zero starvation rate—the mechanism behind the live LCB gap. D14

[†]Canonical artifact: `compose_live_v3_d18_floor400_n2000.json`.

Table 4: Per-benchmark budget share and starvation (allocation-only; $n=200$ batches).

Allocator	Starve rate	LCB share	SWE share	Hard zero-frac
hbac_d18	0.00	15.4%	7.7%	0.00
hbac_guardrail	0.00	15.4%	7.7%	0.00
Type-prior	1.00	0.1%	0.1%	0.67
Uniform	0.00	40.0%	20.0%	0.00

ROI-skip illustrates how optimization pressure can drop entire task classes—a fairness failure mode, not merely poor ML.

8 Limitations

(1) SWE live pass@1 is 0% on 7B and 32B pilot; live claims scoped to LCB+ τ +toolbench. (2) +1.3 ppaggregate gap is LCB-only; D18 $\equiv$ joint $\equiv$ guardrail on 7B live. (3) Oracle–live disconnect: type-prior ties on oracle but starves LCB live. (4) DPO v2 contaminated on LCB; holdout LoRA trained (LCB overlap=0); holdout live matrix pending. (5) L2 frozen ReAct limits joint hierarchical RL claims.

9 Conclusion

Learned batch allocation **matters conditionally**: oracle gains over Tier-A baselines are real but matched by type-prior; live edges are small, paired, and localized where starvation binds under live generation. D18 shows fairness can be reward-learned without inference guardrails on oracle. We release the harness, preregistered analysis, theory appendix, and locked artifacts to support honest extension—including negative results when allocators do not beat simple heuristics.

Reproducibility. `scripts/reproduce_v3.sh` runs CPU analyses; GPU steps via `scripts/rivanna/submit_v3_d` and `submit_capability_pilot.sh`. Canonical manifest: `results/canonical_artifacts.json`.

References

- Mahmoud Hamri and Inbal Talgam-Cohen. Zebra: Zero-shot budgeted resource allocation for llm orchestration. *arXiv preprint arXiv:2605.20485*, 2026. ICML 2026 AgenticUQ Workshop.
- Naman Jain et al. Livecodebench: Holistic and contamination free evaluation of llms for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Neharika Jali, Anupam Nayak, and Gauri Joshi. Not all turns are equally hard: Adaptive thinking budgets for efficient multi-turn reasoning. *arXiv preprint arXiv:2604.05164*, 2026.
- Carlos E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues? In *ICLR*, 2024.

Yujia Qin et al. Toolllm: Facilitating llms to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xu Wan et al. The shadow price of reasoning: Economic perspective on optimal budget allocation for llms. *arXiv preprint arXiv:2606.03092*, 2026. ICML 2026.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025.

Renos Zabounidis et al. Re-forc: Adaptive reward prediction for efficient chain-of-thought reasoning. *arXiv preprint arXiv:2511.02130*, 2025. NeurIPS 2025 Efficient Reasoning Workshop.

A Counterfactual Credit: Bias and Variance Bounds

Let $R(s)$ denote the batch reward under allocation schema s . For each task i in batch $\mathcal{Q}$ with n tasks, define the leave-one-out counterfactual

$$\hat{A}_i = R(s) - R^{(-i)}(s),$$

where $R^{(-i)}(s)$ replaces task i 's budget with the uniform allocation while reusing cached rollouts for other tasks (implementation: `hbac/training/credit.py`).

The credit-weighted reward used in GRPO is

$$\tilde{R}_\beta = (1 - \beta) R(s) + \beta \cdot \frac{1}{n} \sum_{i=1}^n (R(s) + \hat{A}_i).$$

Proposition 1 (Bias bound). *For fixed batch $\mathcal{Q}$ and deterministic oracle replay,*

$$|\tilde{R}_\beta - R(s)| \leq \beta \cdot \max_i |\hat{A}_i|.$$

Proof. Let $\bar{A} = \frac{1}{n} \sum_i \hat{A}_i$. Then $\tilde{R}_\beta - R(s) = \beta(\bar{A} + R(s) - R(s)) = \beta\bar{A}$, and $|\bar{A}| \leq \max_i |\hat{A}_i|$. $\square$

Proposition 2 (Variance bound (schema sampling)). *Let $R(s)$ be random only through schema sampling with variance $\text{Var}(R)$. Then*

$$\text{Var}(\tilde{R}_\beta) \leq (1 - \beta)^2 \text{Var}(R) + \beta^2 \cdot \frac{(\max_i |\hat{A}_i|)^2}{n}.$$

Proof. Apply $\text{Var}(aX + bY) \leq a^2 \text{Var}(X) + b^2 \text{Var}(Y)$ with $a = 1 - \beta$, $b = \beta$, and bound the credit term by $(\max |\hat{A}_i|)^2/n$. $\square$

Consistency. As $\beta \rightarrow 0$, $\tilde{R}_\beta \rightarrow R(s)$: the estimator reduces to standard GRPO batch reward. We use $\beta = 0.2$ in training (`phase3.pipeline.py`); empirical sweep in `results/credit_beta_sweep.json`. Counterfactual credit is a **variance-reduction heuristic**, not an unbiased policy gradient (cf. COMA [?]).

B Stub Track (Track A)

Oracle stub curriculum ($n=300$): HBAC 80% vs. uniform/CLEAR 60%; type-prior ties at 80%. Live stub @ floor=400: HBAC 44.3% vs. uniform 16.7% (+27.6 pp); generous floor (600) ties at 44.3%. Proxy CLEAR/ZEBRA/TAB results retained for regression only.

C Compliant Utility Sensitivity

Secondary metric: $U = R \cdot (1 - \text{violation_rate}) - 0.5 \cdot \text{parse_failures}$. Primary tables use raw pass@1, tokens, and violations.

D Artifact Index

Claim	File
Tier-A oracle	<code>v3_real_oracle_matrix.json</code>
D18 ladder	<code>d18_oracle_ladder.json</code>
V3 live $n=2000$ (D18)	<code>compose_live_v3_d18_floor400_n2000.json</code>
Paired stats (D18)	<code>paired_allocator_analysis_v3_d18.json</code>
DPO audit	<code>dpo_contamination_audit.json</code>
Holdout policy	<code>dpo_holdout_policy.json</code>
Credit β sweep	<code>credit_beta_sweep.json</code>
Preregistered plan	<code>research docs/Preregistered Analysis.md</code>